What is C?

C is a general-purpose programming language created by Dennis Ritchie at the Bell Laboratories in 1972.

It is a very popular language, despite being old. The main reason for its popularity is because it is a fundamental language in the field of computer science.

C is strongly associated with UNIX, as it was developed to write the UNIX operating system.

Why Learn C?

- **It is one of the most popular programming languages in the world**
- If you know C, you will have no problem learning other popular programming languages such as Java, Python, C++, C#, etc, as the syntax is similar
- **C is very fast, compared to other programming languages, like Java and Python**
- **C is very versatile; it can be used in both applications and technologies**

Difference between C and C++

- **C++ was developed as an extension of C, and both languages have almost the same syntax**
- **The main difference between C and C++ is that C++ support classes and objects, while C does not**

C Quickstart

Let's create our first C file.

Open Codeblocks and go to File > New > Empty File.

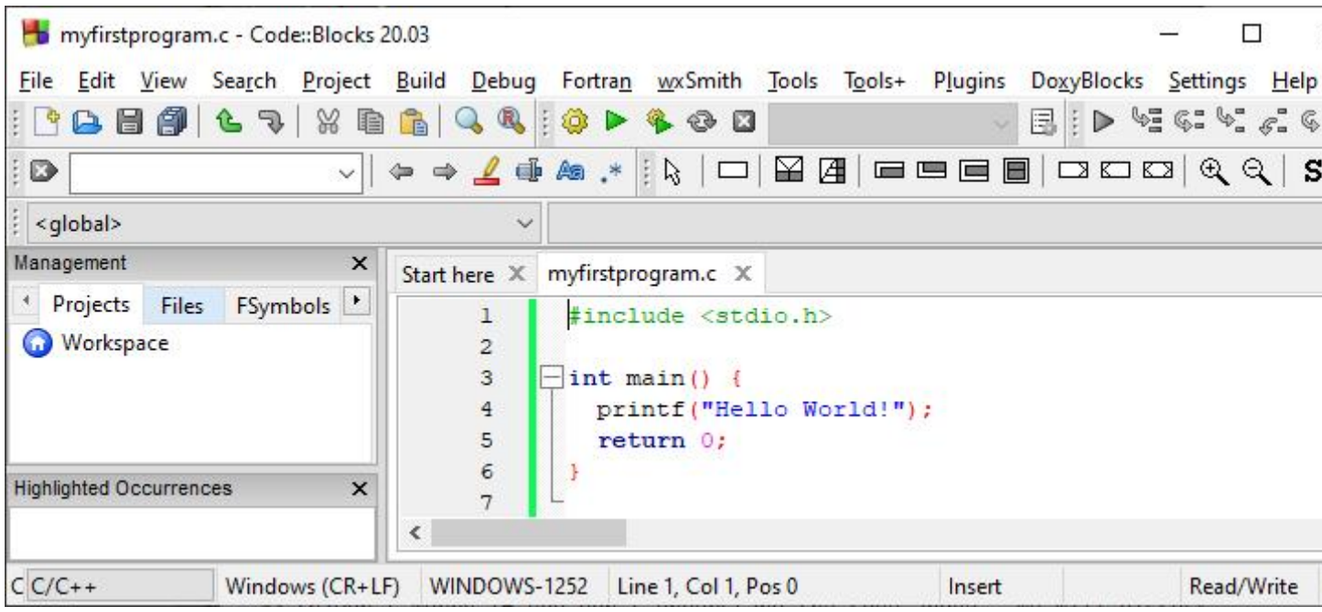
Write the following C code and save the file as myfirstprogram.c (File > Save File as):

myfirstprogram.c

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  return 0;
}
```

In Codeblocks, it should look like this:



Then, go to Build > Build and Run to run (execute) the program. The result will look something to this:

```
Hello World!
Process returned 0 (0x0) execution time : 0.011 s
Press any key to continue.
```

Congratulations! You have now written and executed your first C program.

Syntax

You have already seen the following code a couple of times in the first chapters. Let's break it down to understand it better:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  return 0;
}
```

Try it Yourself »

Example explained

Line 1: #include <stdio.h> is a header file library that lets us work with input and output functions, such as printf() (used in line 4). Header files add functionality to C programs.

Don't worry if you don't understand how #include <stdio.h> works. Just think of it as something that (almost) always appears in your program.

Line 2: A blank line. C ignores white space. But we use it to make the code more readable.

Line 3: Another thing that always appear in a C program is main(). This is called a function. Any code inside its curly brackets {} will be executed.

Line 4: printf() is a function used to output/print text to the screen. In our example, it will output "Hello World!".

Note that: Every C statement ends with a semicolon ;

Note: The body of int main() could also been written as:
int main(){printf("Hello World!");return 0;}

Remember: The compiler ignores white spaces. However, multiple lines makes the code more readable.

Line 5: return 0 ends the main() function.

Line 6: Do not forget to add the closing curly bracket } to actually end the main function.

Statements

A computer program is a list of "instructions" to be "executed" by a computer.

In a programming language, these programming instructions are called statements.

The following statement "instructs" the compiler to print the text "Hello World" to the screen:

Example

```
printf("Hello World!");
```

Try it Yourself »

It is important that you end the statement with a semicolon ;

If you forget the semicolon (;), an error will occur and the program will not run:

Example

```
printf("Hello World!")
```

```
error: expected ';' before 'return'
```

Try it Yourself »

Many Statements

Most C programs contain many statements.

The statements are executed, one by one, in the same order as they are written:

Example

```
printf("Hello World!");
printf("Have a good day!");
return 0;
```

Try it Yourself »

Example explained

From the example above, we have three statements:

1. printf("Hello World!");
2. printf("Have a good day!");
3. return 0;

The first statement is executed first (print "Hello World!" to the screen).
Then the second statement is executed (print "Have a good day!" to the screen).
And at last, the third statement is executed (end the C program successfully).

Output (Print Text)

To output values or print text in C, you can use the printf() function:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  return 0;
}
```

Try it Yourself »

Double Quotes

When you are working with text, it must be wrapped inside double quotations marks "".

If you forget the double quotes, an error occurs:

Example

```
printf("This sentence will work!");


printf(This sentence will produce an error.);
```

Try it Yourself »

Many printf Functions

You can use as many printf() functions as you want. However, note that it does not insert a new line at the end of the output:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  printf("I am learning C.");
  printf("And it is awesome!");
  return 0;
}
```

Try it Yourself »

New Lines

To insert a new line, you can use the \n character:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!\n");
  printf("I am learning C.");
  return 0;
}
```

Try it Yourself »

You can also output multiple lines with a single printf() function. However, this could make the code harder to read:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!\nI am learning C.\nAnd it is awesome!");
  return 0;
}
```

Try it Yourself »

Tip: Two \n characters after each other will create a blank line:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!\n\n");
  printf("I am learning C.");
  return 0;
}
```

Try it Yourself »

What is \n exactly?

The newline character (\n) is called an escape sequence, and it forces the cursor to change its position to the beginning of the next line on the screen. This results in a new line.

Examples of other valid escape sequences are:

Escape Sequence	Description	Try it
\t	Creates a horizontal tab	Try it
\\	Inserts a backslash character (\)	Try it
\"	Inserts a double quote	Try it

character

Comments in C

Comments can be used to explain code, and to make it more readable. It can also be used to prevent execution when testing alternative code.

Comments can be singled-lined or multi-lined.

Single-line Comments

Single-line comments start with two forward slashes (//).

Any text between // and the end of the line is ignored by the compiler (will not be executed).

This example uses a single-line comment before a line of code:

Example

```
// This is a comment
printf("Hello World!");
```

Try it Yourself »

This example uses a single-line comment at the end of a line of code:

Example

```
printf("Hello World!"); // This is a comment
```

Try it Yourself »

C Multi-line Comments

Multi-line comments start with /* and ends with */.

Any text between /* and */ will be ignored by the compiler:

Example

```
/* The code below will print the words Hello World!
to the screen, and it is amazing */
printf("Hello World!");
```

Try it Yourself »

Single or multi-line comments?

It is up to you which you want to use. Normally, we use // for short comments, and /* */ for longer.

Good to know: Before version C99 (released in 1999), you could only use multi-line comments in C.

Variables are containers for storing data values, like numbers and characters.

In C, there are different types of variables (defined with different keywords), for example:

- **int - stores integers (whole numbers), without decimals, such as 123 or -123**
- **float - stores floating point numbers, with decimals, such as 19.99 or -19.99**
- **char - stores single characters, such as 'a' or 'B'. Characters are surrounded by single quotes**

Declaring (Creating) Variables

To create a variable, specify the type and assign it a value:

Syntax

***type variableName* = *value*;**

Where *type* is one of C types (such as int), and *variableName* is the name of the variable (such as x or myName). The equal sign is used to assign a value to the variable.

So, to create a variable that should store a number, look at the following example:

Example

Create a variable called myNum of type int and assign the value 15 to it:

```
int myNum = 15;
```

You can also declare a variable without assigning the value, and assign the value later:

Example

```
// Declare a variable
int myNum;

// Assign a value to the variable
myNum = 15;
```

Output Variables

You learned from the output chapter that you can output values/print text with the printf() function:

Example

```
printf("Hello World!");
```

Try it Yourself »

In many other programming languages (like Python, Java, and C++), you would normally use a print function to display the value of a variable. However, this is not possible in C:

Example

```
int myNum = 15;
printf(myNum);  // Nothing happens
```

Try it Yourself »

To output variables in C, you must get familiar with something called "format specifiers", which you will learn about in the next chapter.

Format Specifiers

Format specifiers are used together with the printf() function to tell the compiler what type of data the variable is storing. It is basically a placeholder for the variable value.

A format specifier starts with a percentage sign %, followed by a character.

For example, to output the value of an int variable, use the format specifier %d surrounded by double quotes (""), inside the printf() function:

Example

```
int myNum = 15;
printf("%d", myNum);  // Outputs 15
```

Try it Yourself »

To print other types, use %c for char and %f for float:

Example

```
// Create variables
int myNum = 15;          // Integer (whole number)
float myFloatNum = 5.99;   // Floating point number
char myLetter = 'D';      // Character
```

```
// Print variables
printf("%d\n", myNum);
printf("%f\n", myFloatNum);
printf("%c\n", myLetter);
```

Try it Yourself »

To combine both text and a variable, separate them with a comma inside the printf() function:

Example

```
int myNum = 15;
printf("My favorite number is: %d", myNum);
```

Try it Yourself »

To print different types in a single printf() function, you can use the following:

Example

```
int myNum = 15;
char myLetter = 'D';
printf("My number is %d and my letter is %c", myNum, myLetter);
```

Try it Yourself »

You will learn more about Data Types in a later chapter.

Print Values Without Variables

You can also just print a value without storing it in a variable, as long as you use the correct format specifier:

Example

```
printf("My favorite number is: %d", 15);
printf("My favorite letter is: %c", 'D');
```

Try it Yourself »

However, it is more sustainable to use variables as they are saved for later and can be re-used whenever.

Change Variable Values

If you assign a new value to an existing variable, it will overwrite the previous value:

Example

```
int myNum = 15;  // myNum is 15
myNum = 10;  // Now myNum is 10
```

Try it Yourself »

You can also assign the value of one variable to another:

Example

```
int myNum = 15;

int myOtherNum = 23;

// Assign the value of myOtherNum (23) to myNum
myNum = myOtherNum;

// myNum is now 23, instead of 15
printf("%d", myNum);
```

Try it Yourself »

Or copy values to empty variables:

Example

```
// Create a variable and assign the value 15 to it
int myNum = 15;

// Declare a variable without assigning it a value
int myOtherNum;

// Assign the value of myNum to myOtherNum
myOtherNum = myNum;

// myOtherNum now has 15 as a value
printf("%d", myOtherNum);
```

Try it Yourself »

Add Variables Together

To add a variable to another variable, you can use the + operator:

Example

```
int x = 5;
int y = 6;
int sum = x + y;
printf("%d", sum);
```

Try it Yourself »

Declare Multiple Variables

To declare more than one variable of the same type, use a comma-separated list:

Example

```
int x = 5, y = 6, z = 50;
printf("%d", x + y + z);
```

Try it Yourself »

You can also assign the same value to multiple variables of the same type:

Example

```
int x, y, z;
x = y = z = 50;
printf("%d", x + y + z);
```

Real-Life Example

Often in our examples, we simplify variable names to match their data type (myInt or myNum for int types, myChar for char types, and so on). This is done to avoid confusion.

However, for a practical example of using variables, we have created a program that stores different data about a college student:

Example

```
// Student data
int studentID = 15;
int studentAge = 23;
float studentFee = 75.25;
char studentGrade = 'B';

// Print variables
printf("Student id: %d\n", studentID);
printf("Student age: %d\n", studentAge);
printf("Student fee: %f\n", studentFee);
printf("Student grade: %c", studentGrade);
```

Try it Yourself »

Calculate the Area of a Rectangle

In this real-life example, we create a program to calculate the area of a rectangle (by multiplying the length and width):

Example

```
// Create integer variables
int length = 4;
int width = 6;
int area;
```

```
// Calculate the area of a rectangle
area = length * width;

// Print the variables
printf("Length is: %d\n", length);
printf("Width is: %d\n", width);
printf("Area of the rectangle is: %d", area);
```

Try it Yourself »

Data Types

As explained in the Variables chapter, a variable in C must be a specified data type, and you must use a format specifier inside the printf() function to display it:

Example

```
// Create variables
int myNum = 5;           // Integer (whole number)
float myFloatNum = 5.99;   // Floating point number
char myLetter = 'D';      // Character

// Print variables
printf("%d\n", myNum);
printf("%f\n", myFloatNum);
printf("%c\n", myLetter);
```

Try it Yourself »

Basic Data Types

The data type specifies the size and type of information the variable will store.

In this tutorial, we will focus on the most basic ones:

Data Type	Size	Description	Example
int	2 or 4 bytes	Stores whole numbers, without decimals	1
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits	1.99
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits	1.99
char	1 byte	Stores a single character/letter/number, or ASCII values	'A'

Basic Format Specifiers

There are different format specifiers for each data type. Here are some of them:

Format Specifier	Data Type	Try it
%d or %i	int	Try it »
%f or %F	float	Try it »
%lf	double	Try it »
%c	char	Try it »
%s	Used for strings (text), which you will learn more about in a later chapter	Try it »

Note: It is important that you use the correct format specifier for the specified data type, or the program may produce errors or even crash.

The char Type

The char data type is used to store a single character.

The character must be surrounded by single quotes, like 'A' or 'c', and we use the %c format specifier to print it:

Example

```
char myGrade = 'A';
printf("%c", myGrade);
```

Try it Yourself »

Alternatively, if you are familiar with ASCII, you can use ASCII values to display certain characters. Note that these values are not surrounded by quotes (''), as they are numbers:

Example

```
char a = 65, b = 66, c = 67;
printf("%c", a);
printf("%c", b);
printf("%c", c);
```

Try it Yourself »

Tip: A list of all ASCII values can be found in our ASCII Table Reference.

Notes on Characters

If you try to store more than a single character, it will only print the last character:

Example

```
char myText = 'Hello';
printf("%c", myText);
```

Try it Yourself »

Note: Don't use the char type for storing multiple characters, as it may produce errors.

To store multiple characters (or whole words), use strings (which you will learn more about in a later chapter):

Example

```
char myText[] = "Hello";
printf("%s", myText);
```

Try it Yourself »

For now, just know that we use strings for storing multiple characters/text, and the char type for single characters

Numeric Types

Use int when you need to store a whole number without decimals, like 35 or 1000, and float or double when you need a floating point number (with decimals), like 9.99 or 3.14515.

int

```
int myNum = 1000;
printf("%d", myNum);
```

Try it Yourself »

float

```
float myNum = 5.75;
printf("%f", myNum);
```

Try it Yourself »

double

```
double myNum = 19.99;
printf("%lf", myNum);
```

Try it Yourself »

float vs. double

The precision of a floating point value indicates how many digits the value can have after the decimal point. The precision of float is six or seven decimal digits, while double variables have a precision of about 15 digits. Therefore, it is often safer to use double for most calculations - but note that it takes up twice as much memory as float (8 bytes vs. 4 bytes).

Scientific Numbers

A floating point number can also be a scientific number with an "e" to indicate the power of 10:

Example

```
float f1 = 35e3;
double d1 = 12E4;

printf("%f\n", f1);
printf("%lf", d1);
```

Try it Yourself »

Set Decimal Precision

You have probably already noticed that if you print a floating point number, the output will show many digits after the decimal point:

Example

```
float myFloatNum = 3.5;
double myDoubleNum = 19.99;
```

Get the Memory Size

We introduced in the data types chapter that the memory size of a variable

Data Type	Size
int	2 or 4 bytes
float	4 bytes
double	8 bytes
char	1 byte

The memory size refers to how much space a type occupies in the *computer's memory*.

To actually get the size (in bytes) of a data type or variable, use the sizeof operator:

Example

```
int myInt;
float myFloat;
double myDouble;
char myChar;

printf("%lu\n", sizeof(myInt));
printf("%lu\n", sizeof(myFloat));
printf("%lu\n", sizeof(myDouble));
printf("%lu\n", sizeof(myChar));
```

Try it Yourself »

Note that we use the %lu format specifer to print the result, instead of %d. It is because the compiler expects the sizeof operator to return a long unsigned int (%lu), instead of int (%d). On some computers it might work with %d, but it is safer to use %lu.

Why Should I Know the Size of Data Types?

Knowing the size of different data types is important because it says something about memory usage and performance.

For example, the size of a char type is 1 byte. Which means if you have an array of 1000 char values, it will occupy 1000 bytes (1 KB) of memory.

Using the right data type for the right purpose will save memory and improve the performance of your program.

You will learn more about the sizeof operator later in this tutorial, and how to use it in different scenarios.

```
printf("%f\n", myFloatNum); // Outputs 3.500000
printf("%lf", myDoubleNum); // Outputs 19.990000
```

Try it Yourself »

If you want to remove the extra zeros (set decimal precision), you can use a dot (.) followed by a number that specifies how many digits that should be shown after the decimal point:

Example

```
float myFloatNum = 3.5;

printf("%f\n", myFloatNum);   // Default will show 6 digits after the decimal point
printf("%.1f\n", myFloatNum); // Only show 1 digit
printf("%.2f\n", myFloatNum); // Only show 2 digits
printf("%.4f", myFloatNum);   // Only show 4 digits
```

Try it Yourself »

Real-Life Example

Here's a real-life example of using different data types, to calculate and output the total cost of a number of items:

Example

```
// Create variables of different data types
int items = 50;
float cost_per_item = 9.99;
float total_cost = items * cost_per_item;
char currency = '$';

// Print variables
printf("Number of items: %d\n", items);
printf("Cost per item: %.2f %c\n", cost_per_item, currency);
printf("Total cost = %.2f %c\n", total_cost, currency);
```

Try it Yourself »

Type Conversion

Sometimes, you have to convert the value of one data type to another type. This is known as type conversion.

For example, if you try to divide two integers, 5 by 2, you would expect the result to be 2.5. But since we are working with integers (and not floating-point values), the following example will just output 2:

Example

```
int x = 5;
int y = 2;
int sum = 5 / 2;

printf("%d", sum); // Outputs 2
```

Try it Yourself »

To get the right result, you need to know how type conversion works.

There are two types of conversion in C:

- Implicit Conversion (automatically)
- Explicit Conversion (manually)

Implicit Conversion

Implicit conversion is done automatically by the compiler when you assign a value of one type to another.

For example, if you assign an int value to a float type:

Example

```
// Automatic conversion: int to float
float myFloat = 9;

printf("%f", myFloat); // 9.000000
```

Try it Yourself »

As you can see, the compiler automatically converts the int value 9 to a float value of 9.000000.

This can be risky, as you might lose control over specific values in certain situations.

Especially if it was the other way around - the following example automatically converts the float value 9.99 to an int value of 9:

Example

```
// Automatic conversion: float to int
int myInt = 9.99;

printf("%d", myInt); // 9
```

Try it Yourself »

What happened to .99? We might want that data in our program! So be careful. It is important that you know how the compiler work in these situations, to avoid unexpected results.

As another example, if you divide two integers: 5 by 2, you know that the sum is 2.5. And as you know from the beginning of this page, if you store the sum as an integer, the result will only display the number 2. Therefore, it would be better to store the sum as a float or a double, right?

Example

```
float sum = 5 / 2;

printf("%f", sum); // 2.000000
```

Try it Yourself »

Why is the result 2.00000 and not 2.5? Well, it is because 5 and 2 are still integers in the division. In this case, you need to manually convert the integer values to floating-point values. (see below).

Explicit Conversion

Explicit conversion is done manually by placing the type in parentheses () in front of the value.

Considering our problem from the example above, we can now get the right result:

Example

```
// Manual conversion: int to float
float sum = (float) 5 / 2;

printf("%f", sum); // 2.500000
```

Try it Yourself »

You can also place the type in front of a variable:

Example

```
int num1 = 5;
int num2 = 2;
float sum = (float) num1 / num2;

prints("%f", sum); // 2.500000
```

Try it Yourself »

And since you learned about "decimal precision" in the previous chapter, you could make the output even cleaner by removing the extra zeros (if you like):

Example

```
int num1 = 5;
int num2 = 2;
float sum = (float) num1 / num2;

printf("%.1f", sum); // 2.5
```

Try it Yourself »

Real-Life Example

Here's a real-life example of data types and type conversion where we create a program to calculate the percentage of a user's score in relation to the maximum score in a game:

Example

```
// Set the maximum possible score in the game to 500
int maxScore = 500;

// The actual score of the user
int userScore = 423;

/* Calculate the percantage of the user's score in relation to the maximum
available score.
Convert userScore to float to make sure that the division is accurate */
float percentage = (float) userScore / maxScore * 100.0;

// Print the percentage
printf("User's percentage is %.2f", percentage);
```

Basic Format Specifiers

There are different format specifiers for each data type. Here are some of them:

Format Specifier	Data Type	Try it
`%d` or `%i`	`int`	Try it »
`%f` or `%F`	`float`	Try it »
`%lf`	`double`	Try it »
`%c`	`char`	Try it »
`%s`	Used for **strings** **(text)**, which you will learn more about in a later chapter	Try it »

Note: It is important that you use the correct format specifier for the specified data type, or the program may produce errors or even crash.

C Character Data Types

❮ PreviousNext ❯

The char Type

The `char` data type is used to store a **single** character.

The character must be surrounded by single quotes, like 'A' or 'c', and we use the `%c` format specifier to print it:

Example

```
char myGrade = 'A';
printf("%c", myGrade);
```

Try it Yourself »

Alternatively, if you are familiar with ASCII, you can use ASCII values to display certain characters. Note that these values are not surrounded by quotes (''), as they are numbers:

Example

```
char a = 65, b = 66, c = 67;
printf("%c", a);
printf("%c", b);
printf("%c", c);
```

Try it Yourself »

Tip: A list of all ASCII values can be found in our ASCII Table Reference.

Notes on Characters

If you try to store more than a single character, it will only print the last character:

Example

```
char myText = 'Hello';
printf("%c", myText);
```

Try it Yourself »

Note: Don't use the `char` type for storing multiple characters, as it may produce errors.

To store multiple characters (or whole words), use strings (which you will learn more about in a later chapter):

Example

```
char myText[] = "Hello";
printf("%s", myText);
```

Try it Yourself »

For now, just know that we use strings for storing multiple characters/text, and the char type for single characters.

C Numeric Data Types

❮ PreviousNext ❯

Numeric Types

Use `int` when you need to store a whole number without decimals, like 35 or 1000, and `float` or `double` when you need a floating point number (with decimals), like 9.99 or 3.14515.

int

```
int myNum = 1000;
printf("%d", myNum);
```

Try it Yourself »

float

```
float myNum = 5.75;
printf("%f", myNum);
```

Try it Yourself »

double

```
double myNum = 19.99;
printf("%lf", myNum);
```

Try it Yourself »

`float` vs. `double`

The **precision** of a floating point value indicates how many digits the value can have after the decimal point. The precision of `float` is six or seven decimal digits, while `double` variables have a precision of about 15 digits. Therefore, it is often safer to use `double` for most calculations - but note that it takes up twice as much memory as `float` (8 bytes vs. 4 bytes).

Scientific Numbers

A floating point number can also be a scientific number with an "e" to indicate the power of 10:

Example

```
float f1 = 35e3;
double d1 = 12E4;
```

```
printf("%f\n", f1);
printf("%lf", d1);
```

Try it Yourself »

Set Decimal Precision

You have probably already noticed that if you print a floating point number, the output will show many digits after the decimal point:

Example

```
float myFloatNum = 3.5;
double myDoubleNum = 19.99;

printf("%f\n", myFloatNum); // Outputs 3.500000
printf("%lf", myDoubleNum); // Outputs 19.990000
```

Try it Yourself »

If you want to remove the extra zeros (set decimal precision), you can use a dot (.) followed by a number that specifies how many digits that should be shown after the decimal point:

Example

```
float myFloatNum = 3.5;

printf("%f\n", myFloatNum);   // Default will show 6 digits after the
decimal point
printf("%.1f\n", myFloatNum); // Only show 1 digit
printf("%.2f\n", myFloatNum); // Only show 2 digits
printf("%.4f", myFloatNum);   // Only show 4 digits
```

Try it Yourself »

C Exercises

Exercise:

Make the output of the following example to only show **one digit** after the decimal point:

```
float myScore = 9.5;
printf("", myScore);
```

C The sizeof Operator

❮ PreviousNext ❯

Get the Memory Size

We introduced in the data types chapter that the memory size of a variable varies depending on the type:

Data Type	Size
`int`	2 or 4 bytes
`float`	4 bytes
`double`	8 bytes
`char`	1 byte

The memory size refers to how much space a type occupies in the *computer's memory*.

To actually get the size (in bytes) of a data type or variable, use the `sizeof` operator:

Example

```
int myInt;
float myFloat;
double myDouble;
char myChar;

printf("%lu\n", sizeof(myInt));
printf("%lu\n", sizeof(myFloat));
printf("%lu\n", sizeof(myDouble));
printf("%lu\n", sizeof(myChar));
```

Try it Yourself »

Note that we use the `%lu` format specifer to print the result, instead of `%d`. It is because the compiler expects the sizeof operator to return a `long unsigned int` (`%lu`), instead of `int` (`%d`). On some computers it might work with `%d`, but it is safer to use `%lu`.

Why Should I Know the Size of Data Types?

Knowing the size of different data types is important because it says something about memory usage and performance.

For example, the size of a `char` type is 1 byte. Which means if you have an array of 1000 `char` values, it will occupy 1000 bytes (1 KB) of memory.

Using the right data type for the right purpose will **save memory** and **improve the performance** of your program.

You will learn more about the `sizeof` operator later in this tutorial, and how to use it in different scenarios.

C Data Types Examples

❮ PreviousNext ❯

Real-Life Example

Here's a real-life example of using different data types, to calculate and output the total cost of a number of items:

Example

```
// Create variables of different data types
int items = 50;
float cost_per_item = 9.99;
float total_cost = items * cost_per_item;
char currency = '$';

// Print variables
printf("Number of items: %d\n", items);
printf("Cost per item: %.2f %c\n", cost_per_item, currency);
printf("Total cost = %.2f %c\n", total_cost, currency);
```

Try it Yourself »

C Type Conversion

❮ PreviousNext ❯

Type Conversion

Sometimes, you have to convert the value of one data type to another type. This is known as **type conversion**.

For example, if you try to divide two integers, `5` by `2`, you would expect the result to be `2.5`. But since we are working with integers (and not floating-point values), the following example will just output `2`:

Example

```
int x = 5;
int y = 2;
int sum = 5 / 2;

printf("%d", sum); // Outputs 2
```

Try it Yourself »

To get the right result, you need to know how **type conversion** works.

There are two types of conversion in C:

- **Implicit Conversion** (automatically)
- **Explicit Conversion** (manually)

Implicit Conversion

Implicit conversion is done automatically by the compiler when you assign a value of one type to another.

For example, if you assign an `int` value to a `float` type:

Example

```
// Automatic conversion: int to float
float myFloat = 9;

printf("%f", myFloat); // 9.000000
```

Try it Yourself »

As you can see, the compiler automatically converts the int value `9` to a float value of `9.000000`.

This can be risky, as you might lose control over specific values in certain situations.

Especially if it was the other way around - the following example automatically converts the float value `9.99` to an int value of `9`:

Example

```
// Automatic conversion: float to int
int myInt = 9.99;

printf("%d", myInt); // 9
```

Try it Yourself »

What happened to `.99`? We might want that data in our program! So be careful. It is important that you know how the compiler work in these situations, to avoid unexpected results.

As another example, if you divide two integers: `5` by `2`, you know that the sum is `2.5`. And as you know from the beginning of this page, if you store the sum as an integer, the result will only display the number `2`. Therefore, it would be better to store the sum as a `float` or a `double`, right?

Example

```
float sum = 5 / 2;

printf("%f", sum); // 2.000000
```

Try it Yourself »

Why is the result `2.00000` and not `2.5`? Well, it is because 5 and 2 are still integers in the division. In this case, you need to manually convert the integer values to floating-point values. (see below).

Explicit Conversion

Explicit conversion is done manually by placing the type in parentheses `()` in front of the value.

Considering our problem from the example above, we can now get the right result:

Example

```
// Manual conversion: int to float
float sum = (float) 5 / 2;

printf("%f", sum); // 2.500000
```

Try it Yourself »

You can also place the type in front of a variable:

Example

```
int num1 = 5;
int num2 = 2;
float sum = (float) num1 / num2;

printf("%f", sum); // 2.500000
```

Try it Yourself »

And since you learned about "decimal precision" in the previous chapter, you could make the output even cleaner by removing the extra zeros (if you like):

Example

```
int num1 = 5;
int num2 = 2;
float sum = (float) num1 / num2;

printf("%.1f", sum); // 2.5
```

Try it Yourself »

Real-Life Example

Here's a real-life example of data types and type conversion where we create a program to calculate the percentage of a user's score in relation to the maximum score in a game:

Example

```
// Set the maximum possible score in the game to 500
int maxScore = 500;

// The actual score of the user
int userScore = 423;

/* Calculate the percantage of the user's score in relation to the maximum
available score.
Convert userScore to float to make sure that the division is accurate */
float percentage = (float) userScore / maxScore * 100.0;

// Print the percentage
printf("User's percentage is %.2f", percentage);
```

Try it Yourself »

C Exercises

Exercise:

Use **type conversion** to make sure that the **result** of the following example is `1.5`, and not just `1`.

```
float sum =  3 / 2;
printf("%.1f", sum);
```

C Constants

❮ PreviousNext ❯

Constants

If you don't want others (or yourself) to change existing variable values, you can use the `const` keyword.

This will declare the variable as "constant", which means **unchangeable** and **read-only**:

Example

```
const int myNum = 15;  // myNum will always be 15
myNum = 10;  // error: assignment of read-only variable 'myNum'
```

Try it Yourself »

You should always declare the variable as constant when you have values that are unlikely to change:

Example

```
const int minutesPerHour = 60;
const float PI = 3.14;
```

Try it Yourself »

Notes On Constants

When you declare a constant variable, it must be assigned with a value:

Example

Like this:

```
const int minutesPerHour = 60;
```

This however, **will not work**:

```
const int minutesPerHour;
minutesPerHour = 60; // error
```

Try it Yourself »

Good Practice

Another thing about constant variables, is that it is considered good practice to declare them with uppercase.

It is not required, but useful for code readability and common for C programmers:

Example

```
const int BIRTHYEAR = 1980;
```

Try it Yourself »

C Exercises

Exercise:

Make sure that the value of the following variable is not possible to change:

```
int hoursPerDay = 24;
```

C Operators

‹ PreviousNext ›

Operators

Operators are used to perform operations on variables and values.

In the example below, we use the + **operator** to add together two values:

Example

```
int myNum = 100 + 50;
```

Try it Yourself »

Although the + operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable:

Example

```
int sum1 = 100 + 50;         // 150 (100 + 50)
int sum2 = sum1 + 250;       // 400 (150 + 250)
int sum3 = sum2 + sum2;      // 800 (400 + 400)
```

Try it Yourself »

C divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Operator	Name	Description	Example	Try it
+	Addition	Adds together two values	x + y	Try it »
-	Subtraction	Subtracts one value from another	x - y	Try it »
*	Multiplication	Multiplies two values	x * y	Try it »

/	Division	Divides one value by another	x / y	Try it »
%	Modulus	Returns the division remainder	x % y	Try it »
++	Increment	Increases the value of a variable by 1	++x	Try it »
--	Decrement	Decreases the value of a variable by 1	--x	Try it »

Assignment Operators

Assignment operators are used to assign values to variables.

In the example below, we use the **assignment** operator (=) to assign the value **10** to a variable called **x**:

Example

```
int x = 10;
```

Try it Yourself »

The **addition assignment** operator (+=) adds a value to a variable:

Example

```
int x = 10;
x += 5;
```

Try it Yourself »

A list of all assignment operators:

Operator	Example	Same As	Try it
=	x = 5	x = 5	Try it »
+=	x += 3	x = x + 3	Try it »

-=	x -= 3	x = x - 3	Try it »
*=	x *= 3	x = x * 3	Try it »
/=	x /= 3	x = x / 3	Try it »
%=	x %= 3	x = x % 3	Try it »
&=	x &= 3	x = x & 3	Try it »
|=	x |= 3	x = x | 3	Try it »
^=	x ^= 3	x = x ^ 3	Try it »
>>=	x >>= 3	x = x >> 3	Try it »
<<=	x <<= 3	x = x << 3	Try it »

ADVERTISEMENT

Comparison Operators

Comparison operators are used to compare two values (or variables). This is important in programming, because it helps us to find answers and make decisions.

The return value of a comparison is either 1 or 0, which means **true** (1) or **false** (0). These values are known as **Boolean values**, and you will learn more about them in the Booleans and If..Else chapter.

In the following example, we use the **greater than** operator (>) to find out if 5 is greater than 3:

Example

```
int x = 5;
int y = 3;
printf("%d", x > y); // returns 1 (true) because 5 is greater than 3
```

Try it Yourself »

A list of all comparison operators:

Operator	Name	Example	Description	Try it
==	Equal to	x == y	Returns 1 if the values are equal	Try it »
!=	Not equal	x != y	Returns 1 if the values are not equal	Try it »
>	Greater than	x > y	Returns 1 if the first value is greater than the second value	Try it »
<	Less than	x < y	Returns 1 if the first value is less than the second value	Try it »
>=	Greater than or equal to	x >= y	Returns 1 if the first value is greater than, or equal to, the second value	Try it »
<=	Less than or equal to	x <= y	Returns 1 if the first value is less than, or equal to, the second value	Try it »

Logical Operators

You can also test for true or false values with logical operators.

Logical operators are used to determine the logic between variables or values, by combining multiple conditions:

Operator	Name	Example	Description	Try it
&&	AND	x < 5 && x < 10	Returns 1 if both statements are true	Try it »
||	OR	x < 5 || x < 4	Returns 1 if one of the statements is true	Try it »
!	NOT	!(x < 5 && x < 10)	Reverse the result, returns 0 if the result is 1	Try it »

C Exercises

Exercise:

Fill in the blanks to multiply `10` with `5`, and print the result:

```
int x = 10;
int y = 5;
printf("", x  y);
```

C Booleans

❮ PreviousNext ❯

Booleans

Very often, in programming, you will need a data type that can only have one of two values, like:

- YES / NO
- ON / OFF
- TRUE / FALSE

For this, C has a `bool` data type, which is known as **booleans**.

Booleans represent values that are either `true` or `false`.

Boolean Variables

In C, the `bool` type is not a built-in data type, like `int` or `char`.

It was introduced in C99, and you must **import** the following header file to use it:

```
#include <stdbool.h>
```

A boolean variable is declared with the `bool` keyword and can take the values `true` or `false`:

```
bool isProgrammingFun = true;
bool isFishTasty = false;
```

Before trying to print the boolean variables, you should know that boolean values are returned as **integers**:

- `1` (or any other number that is not 0) represents `true`
- `0` represents `false`

Therefore, you must use the `%d` format specifier to print a boolean value:

Example

```
// Create boolean variables
bool isProgrammingFun = true;
bool isFishTasty = false;

// Return boolean values
printf("%d", isProgrammingFun);   // Returns 1 (true)
printf("%d", isFishTasty);        // Returns 0 (false)
```

Try it Yourself »

However, it is more common to return a boolean value by **comparing** values and variables.

Comparing Values and Variables

Comparing values are useful in programming, because it helps us to find answers and make decisions.

For example, you can use a comparison operator, such as the **greater than** (`>`) operator, to compare two values:

Example

```
printf("%d", 10 > 9);  // Returns 1 (true) because 10 is greater than 9
```

Try it Yourself »

From the example above, you can see that the return value is a boolean value (1).

You can also compare two variables:

Example

```
int x = 10;
int y = 9;
printf("%d", x > y);
```

Try it Yourself »

In the example below, we use the **equal to** (==) operator to compare different values:

Example

```
printf("%d", 10 == 10); // Returns 1 (true), because 10 is equal to 10
printf("%d", 10 == 15); // Returns 0 (false), because 10 is not equal to
15
printf("%d", 5 == 55);  // Returns 0 (false) because 5 is not equal to 55
```

Try it Yourself »

You are not limited to only compare numbers. You can also compare boolean variables, or even special structures, like arrays (which you will learn more about in a later chapter):

Example

```
bool isHamburgerTasty = true;
bool isPizzaTasty = true;

// Find out if both hamburger and pizza is tasty
printf("%d", isHamburgerTasty == isPizzaTasty);
```

Try it Yourself »

Remember to include the <stdbool.h> header file when working with bool variables.

C Exercises

Exercise:

What is the result of the following example?

```
printf("%d", 15 > 5);
```

C Boolean Examples

❮ PreviousNext ❯

Real Life Example

Let's think of a "real life example" where we need to find out if a person is old enough to vote.

In the example below, we use the `>=` comparison operator to find out if the age (`25`) is **greater than** OR **equal to** the voting age limit, which is set to `18`:

Example

```
int myAge = 25;
int votingAge = 18;

printf("%d", myAge >= votingAge); // Returns 1 (true), meaning 25 year
olds are allowed to vote!
```

Try it Yourself »

Cool, right? An even better approach (since we are on a roll now), would be to wrap the code above in an `if...else` statement, so we can perform different actions depending on the result:

Example

Output "Old enough to vote!" if `myAge` is **greater than or equal to** `18`. Otherwise output "Not old enough to vote.":

```
int myAge = 25;
int votingAge = 18;

if (myAge >= votingAge) {
  printf("Old enough to vote!");
} else {
  printf("Not old enough to vote.");
}
```

Try it Yourself »

Booleans are the basis for all comparisons and conditions.

You will learn more about conditions (`if...else`) in the next chapter.

C If ... Else

❮ PreviousNext ❯

Conditions and If Statements

You have already learned that C supports the usual logical **conditions** from mathematics:

- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`
- Equal to `a == b`
- Not Equal to: `a != b`

You can use these conditions to perform different actions for different decisions.

C has the following conditional statements:

- Use `if` to specify a block of code to be executed, if a specified condition is `true`
- Use `else` to specify a block of code to be executed, if the same condition is `false`
- Use `else if` to specify a new condition to test, if the first condition is `false`
- Use `switch` to specify many alternative blocks of code to be executed

The if Statement

Use the `if` statement to specify a block of code to be executed if a condition is `true`.

Syntax

```
if (condition) {
  // block of code to be executed if the condition is true
}
```

Note that `if` is in lowercase letters. Uppercase letters (If or IF) will generate an error.

In the example below, we test two values to find out if 20 is greater than 18. If the condition is `true`, print some text:

Example

```
if (20 > 18) {
  printf("20 is greater than 18");
}
```

Try it Yourself »

We can also test variables:

Example

```
int x = 20;
int y = 18;
if (x > y) {
  printf("x is greater than y");
}
```

Try it Yourself »

Example explained

In the example above we use two variables, **x** and **y**, to test whether x is greater than y (using the `>` operator). As x is 20, and y is 18, and we know that 20 is greater than 18, we print to the screen that "x is greater than y".

C Exercises

Exercise:

Print "Hello World" if `x` is **greater than** `y`.

```
int x = 50;
int y = 10;
 (x  y) {
  printf("Hello World");
}
```

Submit Answer »

Start the Exercise

C Else

The else Statement

Use the `else` statement to specify a block of code to be executed if the condition is `false`.

Syntax

```
if (condition) {
  // block of code to be executed if the condition is true
} else {
  // block of code to be executed if the condition is false
}
```

Example

```
int time = 20;
if (time < 18) {
  printf("Good day.");
} else {
  printf("Good evening.");
}
// Outputs "Good evening."
```

Try it Yourself »

Example explained

In the example above, time (20) is greater than 18, so the condition is `false`. Because of this, we move on to the `else` condition and print to the screen "Good evening". If the time was less than 18, the program would print "Good day"

he else if Statement

Use the `else if` statement to specify a new condition if the first condition is `false`.

Syntax

```
if (condition1) {
  // block of code to be executed if condition1 is true
} else if (condition2) {
  // block of code to be executed if the condition1 is false and
condition2 is true
} else {
  // block of code to be executed if the condition1 is false and
condition2 is false
}
```

Example

```
int time = 22;
if (time < 10) {
```

```
  printf("Good morning.");
} else if (time < 20) {
  printf("Good day.");
} else {
  printf("Good evening.");
}
// Outputs "Good evening."
```

Try it Yourself »

Example explained

In the example above, time (22) is greater than 10, so the **first condition** is `false`. The next condition, in the `else if` statement, is also `false`, so we move on to the `else` condition since **condition1** and **condition2** is both `false` - and print to the screen "Good evening".

However, if the time was 14, our program would print "Good day."

C Short Hand If Else

❮ PreviousNext ❯

Short Hand If...Else (Ternary Operator)

There is also a short-hand if else, which is known as the **ternary operator** because it consists of three operands. It can be used to replace multiple lines of code with a single line. It is often used to replace simple if else statements:

Syntax

```
variable = (condition) ? expressionTrue : expressionFalse;
```

Instead of writing:

Example

```
int time = 20;
if (time < 18) {
  printf("Good day.");
} else {
  printf("Good evening.");
}
```

Try it Yourself »

You can simply write:

Example

```
int time = 20;
(time < 18) ? printf("Good day.") : printf("Good evening.");
```

Try it Yourself »

It is completely up to you if you want to use the traditional if...else statement or the ternary operator.

Real-Life Examples

This example shows how you can use `if..else` to "open a door" if the user enters the correct code:

Example

```
int doorCode = 1337;

if (doorCode == 1337) {
  printf("Correct code.\nThe door is now open.");
} else {
  printf("Wrong code.\nThe door remains closed.");
}
```

Try it Yourself »

This example shows how you can use `if..else` to find out if a number is positive or negative:

Example

```
int myNum = 10; // Is this a positive or negative number?

if (myNum > 0) {
  printf("The value is a positive number.");
} else if (myNum < 0) {
  printf("The value is a negative number.");
} else {
  printf("The value is 0.");
}
```

Try it Yourself »

Find out if a person is old enough to vote:

Example

```
int myAge = 25;
int votingAge = 18;

if (myAge >= votingAge) {
  printf("Old enough to vote!");
} else {
  printf("Not old enough to vote.");
}
```

Try it Yourself »

Find out if a number is even or odd:

Example

```
int myNum = 5;

if (myNum % 2 == 0) {
  printf("%d is even.\n", myNum);
} else {
  printf("%d is odd.\n", myNum);
}
```

Try it Yourself »

C Switch

❮ PreviousNext ❯

Switch Statement

Instead of writing **many** `if..else` statements, you can use the `switch` statement.

The `switch` statement selects one of many code blocks to be executed:

Syntax

```
switch (expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

This is how it works:

- The `switch` expression is evaluated once
- The value of the expression is compared with the values of each `case`
- If there is a match, the associated block of code is executed
- The `break` statement breaks out of the switch block and stops the execution
- The `default` statement is optional, and specifies some code to run if there is no case match

The example below uses the weekday number to calculate the weekday name:

Example

```
int day = 4;

switch (day) {
  case 1:
    printf("Monday");
    break;
  case 2:
    printf("Tuesday");
    break;
  case 3:
    printf("Wednesday");
    break;
```

```
  case 4:
    printf("Thursday");
    break;
  case 5:
    printf("Friday");
    break;
  case 6:
    printf("Saturday");
    break;
  case 7:
    printf("Sunday");
    break;
}

// Outputs "Thursday" (day 4)
```

Try it Yourself »

The break Keyword

When C reaches a `break` keyword, it breaks out of the switch block.

This will stop the execution of more code and case testing inside the block.

When a match is found, and the job is done, it's time for a break. There is no need for more testing.

A break can save a lot of execution time because it "ignores" the execution of all the rest of the code in the switch block.

The default Keyword

The `default` keyword specifies some code to run if there is no case match:

Example

```
int day = 4;

switch (day) {
  case 6:
    printf("Today is Saturday");
    break;
  case 7:
    printf("Today is Sunday");
    break;
  default:
    printf("Looking forward to the Weekend");
}

// Outputs "Looking forward to the Weekend"
```

Try it Yourself »

Note: The default keyword must be used as the last statement in the switch, and it does not need a break.

C Exercises

Exercise:

Insert the missing parts to complete the following switch statement:

```
int day = 2;
switch () {
 1:
    printf("Monday");
    ;
 2:
    printf("Sunday");
    ;
}
```

C While Loop

‹ PreviousNext ›

Loops

Loops can execute a block of code as long as a specified condition is reached.

Loops are handy because they save time, reduce errors, and they make code more readable.

While Loop

The `while` loop loops through a block of code as long as a specified condition is `true`:

Syntax

```
while (condition) {
  // code block to be executed
}
```

In the example below, the code in the loop will run, over and over again, as long as a variable (i) is less than 5:

Example

```
int i = 0;

while (i < 5) {
  printf("%d\n", i);
  i++;
}
```

Try it Yourself »

Note: Do not forget to increase the variable used in the condition (i++), otherwise the loop will never end!

C Exercises

Exercise:

Print i as long as i is less than 6:

```
int i = 1;
 (i < 6) {
  printf("%d\n", i);
  ;
}
```

C Do/While Loop

‹ PreviousNext ›

The Do/While Loop

The do/while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax

```
do {
  // code block to be executed
}
while (condition);
```

The example below uses a do/while loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested:

Example

```
int i = 0;

do {
  printf("%d\n", i);
  i++;
}
while (i < 5);
```

Try it Yourself »

Do not forget to increase the variable used in the condition, otherwise the loop will never end!

C While Loop Examples

❮ PreviousNext ❯

Real-Life Examples

To demonstrate a practical example of the **while loop**, we have created a simple "countdown" program:

Example

```
int countdown = 3;

while (countdown > 0) {
  printf("%d\n", countdown);
  countdown--;
}
```

```
printf("Happy New Year!!\n");
```

Try it Yourself »

In this example, we create a program that only print even numbers between 0 and 10 (inclusive):

Example

```
int i = 0;

while (i <= 10) {
  printf("%d\n", i);
  i += 2;
}
```

Try it Yourself »

In this example we use a while loop to reverse some numbers:

Example

```
// A variable with some specific numbers
int numbers = 12345;

// A variable to store the reversed number
int revNumbers = 0;

// Reverse and reorder the numbers
while (numbers) {
  // Get the last number of 'numbers' and add it to 'revNumber'
  revNumbers = revNumbers * 10 + numbers % 10;
  // Remove the last number of 'numbers'
  numbers /= 10;
}
```

Try it Yourself »

To demonstrate a practical example of the **while loop** combined with an **if else statement**, let's say we play a game of Yatzy:

Example

Print "Yatzy!" If the dice number is 6:

```
int dice = 1;

while (dice <= 6) {
  if (dice < 6) {
    printf("No Yatzy\n");
  } else {
    printf("Yatzy!\n");
  }
  dice = dice + 1;
}
```

Try it Yourself »

If the loop passes the values ranging from 1 to 5, it prints "No Yatzy".
Whenever it passes the value 6, it prints "Yatzy!".

C For Loop

❮ PreviousNext ❯

For Loop

When you know exactly how many times you want to loop through a block of code, use the `for` loop instead of a `while` loop:

Syntax

```
for (expression 1; expression 2; expression 3) {
  // code block to be executed
}
```

Expression 1 is executed (one time) before the execution of the code block.

Expression 2 defines the condition for executing the code block.

Expression 3 is executed (every time) after the code block has been executed.

The example below will print the numbers 0 to 4:

Example

```
int i;

for (i = 0; i < 5; i++) {
  printf("%d\n", i);
}
```

Try it Yourself »

Example explained

Expression 1 sets a variable before the loop starts (int i = 0).

Expression 2 defines the condition for the loop to run (i must be less than 5). If the condition is true, the loop will start over again, if it is false, the loop will end.

Expression 3 increases a value (i++) each time the code block in the loop has been executed.

C Exercises

Exercise:

Use a **for loop** to print "Yes" 5 times:

```
 (int i = 0; i < 5; ) {
  printf("Yes\n");
}
```

Submit Answer »

Start the Exercise

C Nested Loops

❮ PreviousNext ❯

Nested Loops

It is also possible to place a loop inside another loop. This is called a **nested loop**.

The "inner loop" will be executed one time for each iteration of the "outer loop":

Example

```
int i, j;



// Outer loop

for (i = 1; i <= 2; ++i) {

  printf("Outer: %d\n", i);  // Executes 2 times



  // Inner loop

  for (j = 1; j <= 3; ++j) {

    printf(" Inner: %d\n", j);  // Executes 6 times (2 * 3)

  }
```

}C For Loop Examples

❮ PreviousNext ❯

Real-Life Examples

To demonstrate a practical example of the **for loop**, let's create a program that counts to 100 by tens:

Example

```
for (i = 0; i <= 100; i += 10) {
  printf("%d\n", i);
}
```

Try it Yourself »

In this example, we create a program that only print even numbers between 0 and 10 (inclusive):

Example

```
for (i = 0; i <= 10; i = i + 2) {
  printf("%d\n", i);
}
```

Try it Yourself »

Here we only print odd numbers:

Example

```
for (i = 1; i < 10; i = i + 2) {
  printf("%d\n", i);
}
```

Try it Yourself »

In this example we print the powers of 2 up to 512:

Example

```
for (i = 2; i <= 512; i *= 2) {
  printf("%d\n", i);
}
```

Try it Yourself »

And in this example, we create a program that prints the multiplication table for a specified number:

Example

```
int number = 2;
int i;

// Print the multiplication table for the number 2
for (i = 1; i <= 10; i++) {
  printf("%d x %d = %d\n", number, i, number * i);
}

return 0;
```

C Break and Continue

Break

You have already seen the `break` statement used in an earlier chapter of this tutorial. It was used to "jump out" of a `switch` statement.

The `break` statement can also be used to jump out of a **loop**.

This example jumps out of the **for loop** when `i` is equal to 4:

Example

```
int i;

for (i = 0; i < 10; i++) {
  if (i == 4) {
    break;
  }
  printf("%d\n", i);
}
```

Try it Yourself »

Continue

The `continue` statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of 4:

Example

```
int i;

for (i = 0; i < 10; i++) {
  if (i == 4) {
    continue;
  }
  printf("%d\n", i);
}
```

Try it Yourself »

Break and Continue in While Loop

You can also use `break` and `continue` in while loops:

Break Example

```
int i = 0;

while (i < 10) {
  if (i == 4) {
    break;
  }
  printf("%d\n", i);
  i++;
}
```

Try it Yourself »

Continue Example

```
int i = 0;

while (i < 10) {
  if (i == 4) {
    i++;
    continue;
  }
  printf("%d\n", i);
  i++;
}
```

Try it Yourself »

C Exercises

Exercise:

Stop the loop if `i` is 5.

```
for (int i = 0; i < 10; i++) {
  if (i == 5) {
    ;
  }
  printf("%d\n", i);
}
```

C Arrays

❮ PreviousNext ❯

Arrays

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

To create an array, define the data type (like `int`) and specify the name of the array followed by **square brackets []**.

To insert values to it, use a comma-separated list, inside curly braces:

```
int myNumbers[] = {25, 50, 75, 100};
```

We have now created a variable that holds an array of four integers.

Access the Elements of an Array

To access an array element, refer to its **index number**.

Array indexes start with **0**: [0] is the first element. [1] is the second element, etc.

This statement accesses the value of the **first element [0]** in `myNumbers`:

Example

```
int myNumbers[] = {25, 50, 75, 100};
printf("%d", myNumbers[0]);

// Outputs 25
```

Try it Yourself »

Change an Array Element

To change the value of a specific element, refer to the index number:

Example

```
myNumbers[0] = 33;
```

Example

```
int myNumbers[] = {25, 50, 75, 100};
myNumbers[0] = 33;

printf("%d", myNumbers[0]);

// Now outputs 33 instead of 25
```

Try it Yourself »

ADVERTISEMENT

Loop Through an Array

You can loop through the array elements with the `for` loop.

The following example outputs all elements in the `myNumbers` array:

Example

```
int myNumbers[] = {25, 50, 75, 100};
int i;

for (i = 0; i < 4; i++) {
  printf("%d\n", myNumbers[i]);
}
```

Try it Yourself »

Set Array Size

Another common way to create arrays, is to specify the size of the array, and add elements later:

Example

```
// Declare an array of four integers:
int myNumbers[4];

// Add elements
myNumbers[0] = 25;
```

```
myNumbers[1] = 50;
myNumbers[2] = 75;
myNumbers[3] = 100;
```

Try it Yourself »

Using this method, **you should know the number of array elements in advance,** in order for the program to store enough memory.

You are not able to change the size of the array after creation.

C Exercises

Exercise:

Create an array of type `int` called `myNumbers`.

```
  {25, 50, 75, 100};
```

C Array Size

❮ PreviousNext ❯

Get Array Size or Length

To get the size of an array, you can use the `sizeof` operator:

Example

```
int myNumbers[] = {10, 25, 50, 75, 100};
printf("%lu", sizeof(myNumbers)); // Prints 20
```

Try it Yourself »

Why did the result show `20` instead of `5`, when the array contains 5 elements?

- It is because the `sizeof` operator returns the size of a type in **bytes**.

You learned from the Data Types chapter that an `int` type is usually 4 bytes, so from the example above, 4 x 5 (*4 bytes x 5 elements*) = **20 bytes**.

Knowing the memory size of an array is great when you are working with larger programs that require good memory management.

But when you just want to find out how many elements an array has, you can use the following formula (which divides the size of the array by the size of one array element):

Example

```
int myNumbers[] = {10, 25, 50, 75, 100};
int length = sizeof(myNumbers) / sizeof(myNumbers[0]);

printf("%d", length);  // Prints 5
```

Try it Yourself »

Making Better Loops

In the array loops section in the previous chapter, we wrote the size of the array in the loop condition (`i < 4`). This is not ideal, since it will only work for arrays of a specified size.

However, by using the `sizeof` formula from the example above, we can now make loops that work for arrays of any size, which is more sustainable.

Instead of writing:

Example

```
int myNumbers[] = {25, 50, 75, 100};
int i;

for (i = 0; i < 4; i++) {
  printf("%d\n", myNumbers[i]);
}
```

Try it Yourself »

It is better to write:

Example

```
int myNumbers[] = {25, 50, 75, 100};
int length = sizeof(myNumbers) / sizeof(myNumbers[0]);
int i;

for (i = 0; i < length; i++) {
  printf("%d\n", myNumbers[i]);
```

} C Arrays - Real-Life Examples

❮ PreviousNext ❯

Real-Life Example

To demonstrate a practical example of using arrays, let's create a program that calculates the average of different ages:

Example

```
// An array storing different ages
int ages[] = {20, 22, 18, 35, 48, 26, 87, 70};

float avg, sum = 0;
int i;

// Get the length of the array
int length = sizeof(ages) / sizeof(ages[0]);

// Loop through the elements of the array
for (i = 0; i < length; i++) {
  sum += ages[i];
}

// Calculate the average by dividing the sum by the length
avg = sum / length;

// Print the average
printf("The average age is: %.2f", avg);
```

Try it Yourself »

And in this example, we create a program that finds the lowest age among different ages:

Example

```
// An array storing different ages
int ages[] = {20, 22, 18, 35, 48, 26, 87, 70};

int i;
```

```
// Get the length of the array
int length = sizeof(ages) / sizeof(ages[0]);

// Create a variable and assign the first array element of ages to it
int lowestAge = ages[0];

// Loop through the elements of the ages array to find the lowest age
for (i = 0; i < length; i++) {
  if (lowestAge > ages[i]) {
    lowestAge = ages[i];
  }
}
```